Graf algoritmi

Vježba 2

1. **Visinska karta otoka.** U datoteci `otok.txt` zapisana je matrica visina: `0` je more, a pozitivan broj visina kopna. Rub karte je more.

   (a) Napisati modul `matrica.py` s funkcijom `read_matrix(filename)` koja čita matricu cijelih brojeva iz datoteke (brojevi u retku odvojeni su razmacima) i funkcijom `print_matrix(matrix)` koja je ispisuje. Funkcije importati u rješenju zadatka.

   (b) Napisati funkciju koja vraća broj polja kopna i tuple `(visina, redak, stupac)` najviše točke otoka.

   (c) Polje `(i, j)` ima četiri susjeda: gore, dolje, lijevo i desno. Vrh je polje kopna više od sva četiri susjeda. Napisati funkciju koja vraća listu svih vrhova.

   (d) Polje kopna je obala ako mu je barem jedan susjed more. Napisati funkciju koja vraća broj polja obale i funkciju koja vraća kartu kao listu stringova u kojoj je more `~`, obala `#`, a unutrašnjost `o`.

   Rezultati: Za `otok.txt` rezultati su `(57, (7, 4, 4))`, `[(4, 4), (5, 8), (6, 9)]` i `23`.

2. **Tko koga prati.** Svaki redak datoteke `prati.txt` sadrži dva imena, npr. `ana bruno`, što znači da Ana prati Bruna (obrnuto ne mora vrijediti).

   (a) Napisati funkciju koja čita datoteku i vraća dictionary u kojem je svakoj osobi pridružen set osoba koje ona prati.

   (b) Napisati funkciju koja od tog dictionaryja napravi novi, u kojem je svakoj osobi pridružen set osoba koje **nju** prate.

   (c) Napisati funkciju koja vraća sortiranu listu parova `(a, b)`, `a < b`, koji se prate međusobno.

   (d) Napisati funkcije koje vraćaju osobu s najviše pratitelja (i broj pratitelja), listu osoba koje nitko ne prati te listu osoba koje zadana osoba prati, a one nju ne prate.

   Rezultati: Za `prati.txt` međusobno se prati 6 parova (prvi je `('ana', 'bruno')`), najviše pratitelja ima `ana` (8), nitko ne prati `kate`, a za `'cvita'` zadnja funkcija vraća `['duje', 'ema']`.

3. **Anagrami.** Riječi su anagrami ako se sastoje od istih slova, npr. `listen` i `silent`. U datoteci `rijeci.txt` nalazi se po jedna riječ u retku.

   (a) Napisati funkciju koja vraća dictionary u kojem je ključ string slova riječi poredanih abecedno (za `listen` to je `'eilnst'`), a vrijednost lista riječi s tim slovima.

   (b) Napisati funkciju koja vraća listu grupa anagrama s barem dvije riječi, sortiranu od najveće grupe prema najmanjoj. Grupe iste veličine poredati abecedno.

(c) Napisati funkciju koja za zadanu riječ vraća listu njezinih anagrama iz datoteke (bez same zadane riječi).

Rezultati: Za `rijeci.txt` postoji 12 grupa, a najveća je `['listen', 'silent', 'enlist', 'tinsel', 'inlets']`. Za `'stone'` rezultat je `['notes', 'onset', 'tones']`.

4. **Udaljenosti među gradovima.** Datoteka `distance.csv` sadrži cestovne udaljenosti među 37 europskih gradova u kilometrima. Prvi redak i prvi stupac su imena gradova, a na dijagonali je znak `-`.

   (a) Napisati funkciju koja vraća listu imena gradova i matricu udaljenosti (na dijagonali `0`).

   (b) Napisati funkciju koja od liste imena i matrice napravi ugniježđeni dictionary, tako da je `d['Paris']['Rome']` udaljenost od Pariza do Rima.

   (c) Napisati funkciju koja za zadani grad vraća listu `k` najbližih gradova kao tuple `(udaljenost, grad)`.

   Primjer: za `'Vienna'` i `k = 3` rezultat je

   ```
   [(312, 'Prague'), (458, 'Munich'), (610, 'Venice')]
   ```

   (d) Napisati funkciju koja vraća dva najudaljenija grada i njihovu udaljenost.

   (e) Matrica bi trebala biti simetrična. Napisati funkciju koja vraća listu svih parova gradova za koje udaljenost od `A` do `B` nije jednaka udaljenosti od `B` do `A`.

   Rezultati: Za `distance.csv` najudaljeniji su gradovi `Athens` i `Lisbon` (4578 km), a nesimetričnih parova ima 6, npr. `Genoa` i `Zurich` (461 i 641).

Napomene:

- Rješenja se predaju u obliku `prezime_ime_brojvjezbe_oznakazadatka.py`, a uz njih i modul `matrica.py`.
- Sav kod treba se nalaziti u funkcijama. Koristiti konstrukt:

```python
def main():
    # funkcija iz koje se pozivaju ostale funkcije
    # za razlicite testne primjere

if __name__ == '__main__':
    main()
```